

MANAGING DATA

LEARNING OBJECTIVES

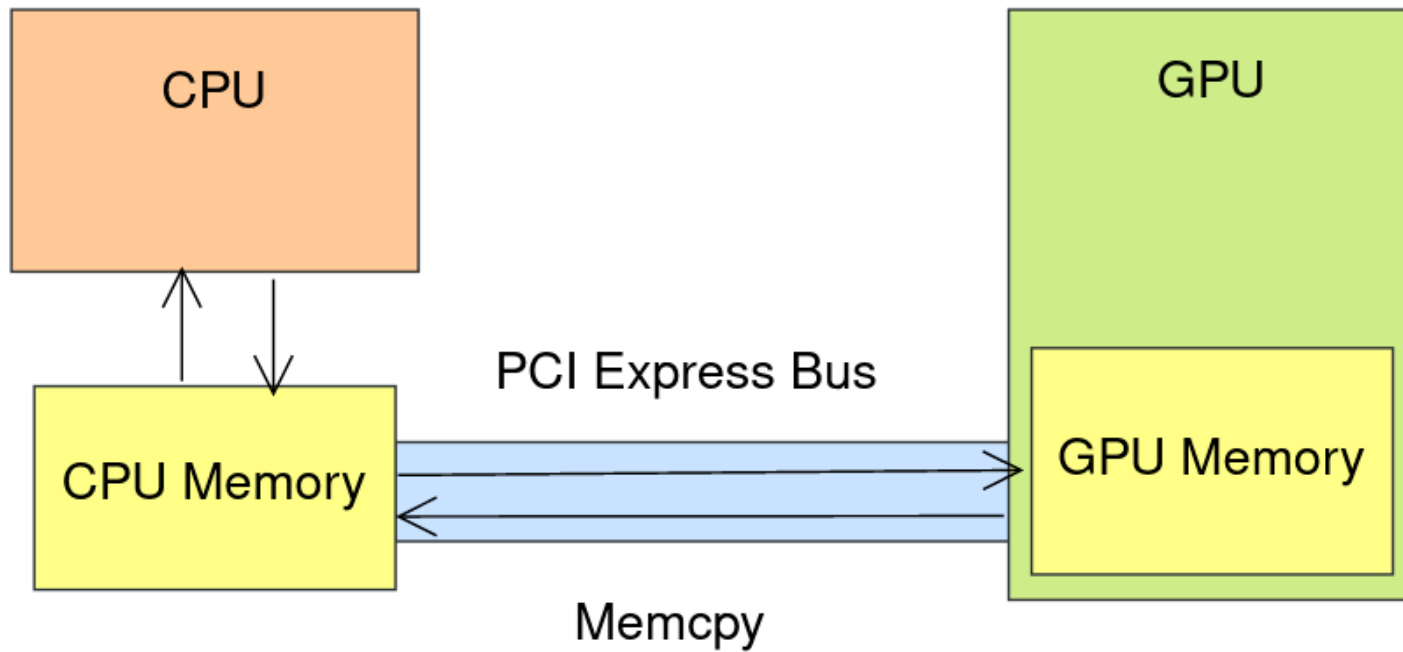
- Learn about the USM data management model
- Learn how to allocate, transfer and free memory using USM.
- Learn how to access data in a kernel function
- Learn about the variations of USM
- Learn how to represent basic data flow graphs

MEMORY MODELS

- In SYCL there are two models for managing data:
 - The buffer/accessor model.
 - The USM (unified shared memory) model ← *we will focus on this.*
- Which model you choose can have an effect on how you enqueue kernel functions.

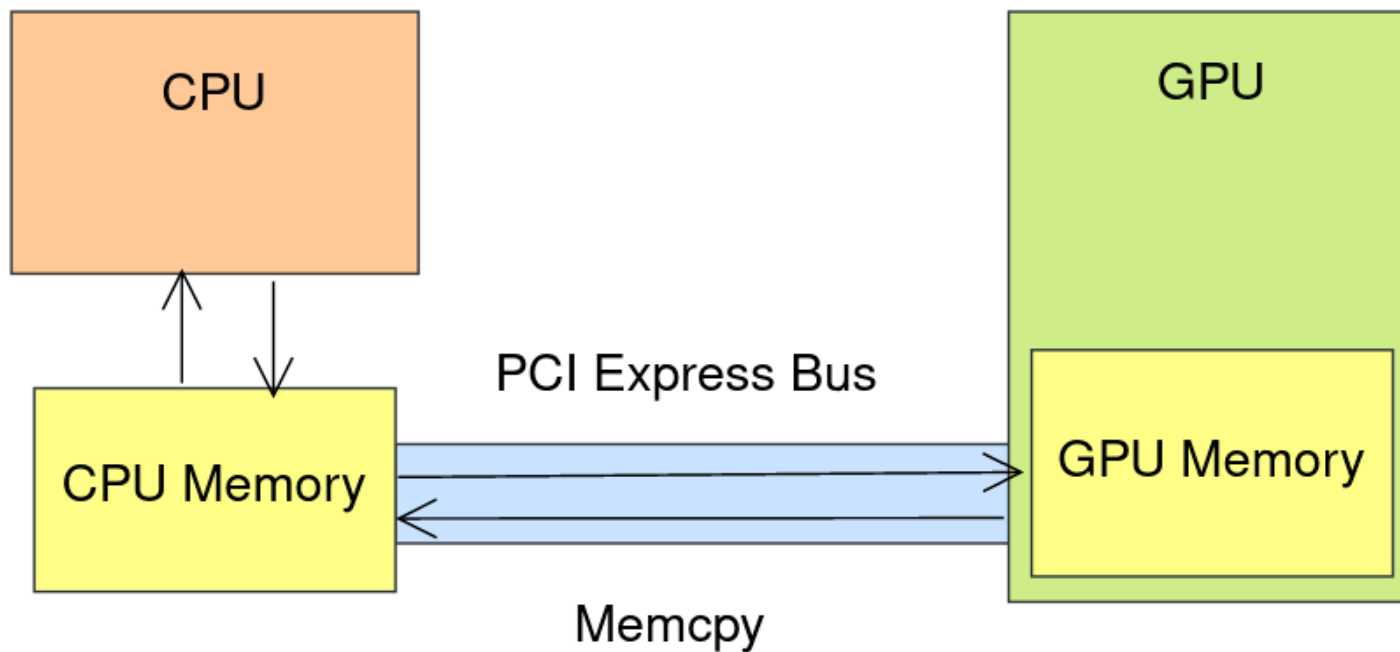
CPU AND GPU MEMORY

- A GPU has its own memory, separate to CPU memory.
- In order for the GPU to use memory from the CPU, the following actions must take place (either explicitly or implicitly):
 - Memory allocation on the GPU.
 - Data migration from the CPU to the allocation on the GPU.
 - Some computation on the GPU.
 - Migration of the result back to the CPU.

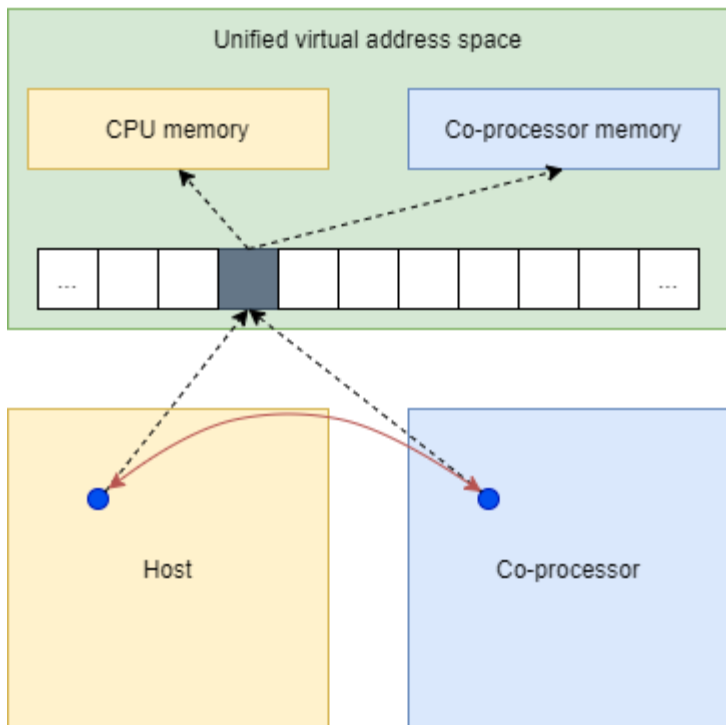


CPU AND GPU MEMORY

- Memory transfers between CPU and GPU are a bottleneck.
- We want to minimize these transfers, when possible.

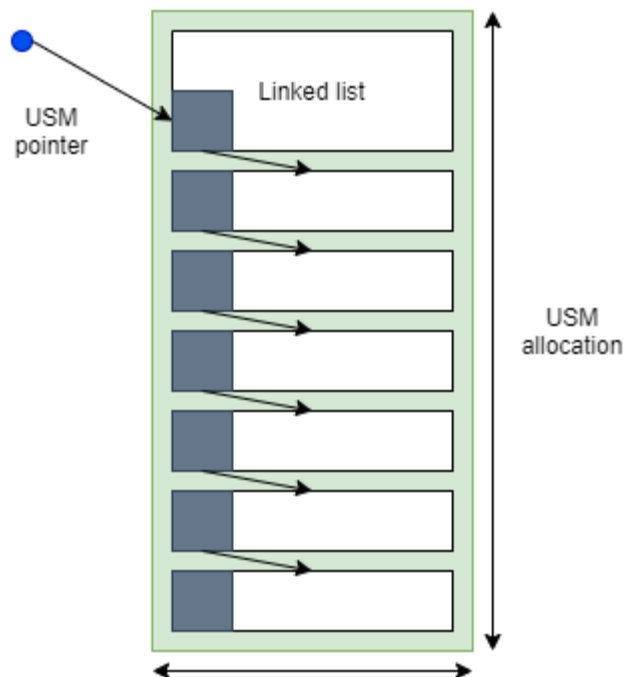


UNIFIED VIRTUAL ADDRESS SPACE



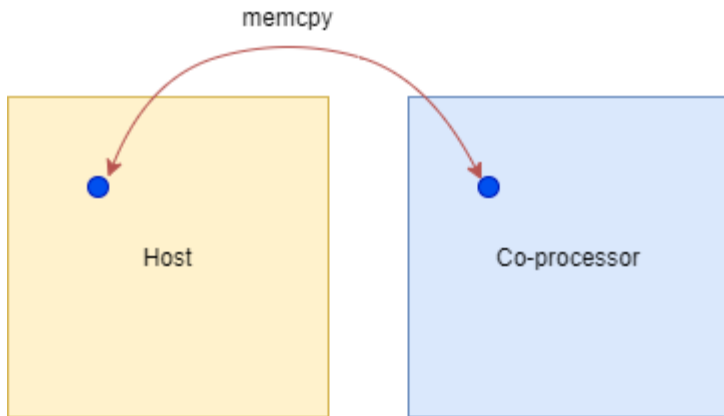
- USM memory allocations return pointers which are consistent between the host application and kernel functions on a device
- Representing data between the host and device(s) does not require creating accessors
- Pointer-based API more familiar to C or C++ programmers

POINTER BASED STRUCTURES



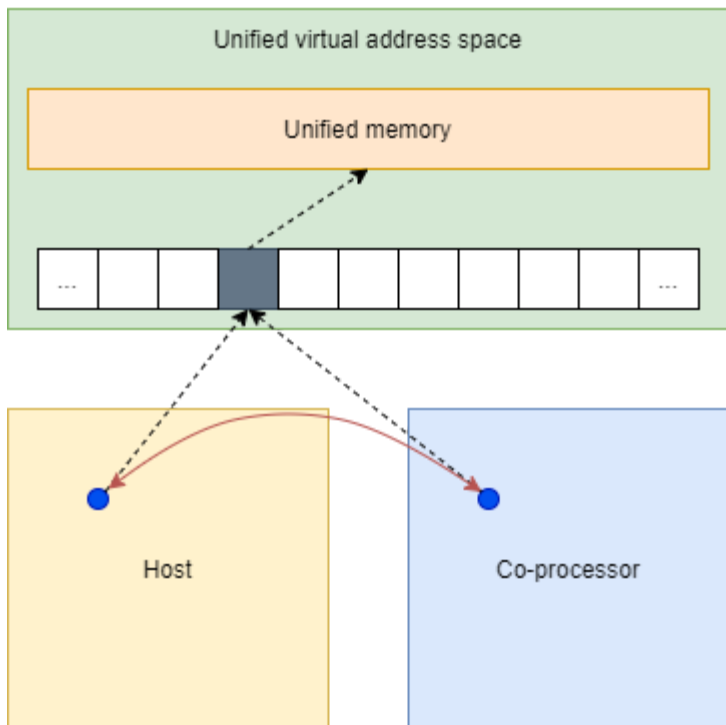
- Data is moved between the host and device(s) in a span of memory in bytes rather than a buffer of a specific type
- Pointers within that region of memory can freely point to any other address in that region
- Easier to port existing C or C++ code to use SYCL

EXPLICIT MEMORY MANAGEMENT



- Memory is allocated and data is moved using explicit routines
- Moving data between the host and device(s) does not require accessors or submitting command groups
- The SYCL runtime will not perform any data dependency analysis, dependencies between commands must be managed manually

SHARED MEMORY ALLOCATIONS



- Some platforms will support variants of USM where memory allocations share the same memory region between the host and device(s)
- No explicit routines are required to move the data between the host and device(s)

USM ALLOCATION TYPES

USM has three different kinds of memory allocation

- A **host** allocation is allocated in host memory
- A **device** allocation is allocation in device memory
- A **shared** allocation is allocated in shared memory and can migrate back and forth

USM VARIANTS

USM has four variants which a platform can support with varying levels of support

	Explicit USM (minimum)	Restricted USM (optional)	Concurrent USM (optional)	System USM (optional)
Consistent pointers	✓	✓	✓	✓
Pointer-based structures	✓	✓	✓	✓
Explicit data movement	✓	✓	✓	✓
Shared memory allocations	✗	✓	✓	✓
Concurrent access	✗	✗	✓	✓
System allocations	✗	✗	✗	✓

QUERYING FOR SUPPORT

Each SYCL platform and its device(s) will support different variants of USM and different kinds of memory allocation

```
if (dev.has(sycl::aspect::usm_device_allocations))
```

USING USM - MALLOC DEVICE

```
// Allocate memory on device
T *device_ptr = sycl::malloc_device<T>(n, myQueue);

// Copy data to device
myQueue.memcpy(device_ptr, cpu_ptr, n * sizeof(T));

// ...
// Do some computation on device
// ...

// Copy data back to CPU
myQueue.memcpy(result_ptr, device_ptr, n * sizeof(T)).wait();

// Free allocated data
sycl::free(device_ptr, myQueue);
```

- It is important to free memory after it has been used to avoid memory leaks.

USING USM - MALLOC SHARED

```
// Allocate shared memory
T *shared_ptr = sycl::malloc_shared<T>(n, myQueue);

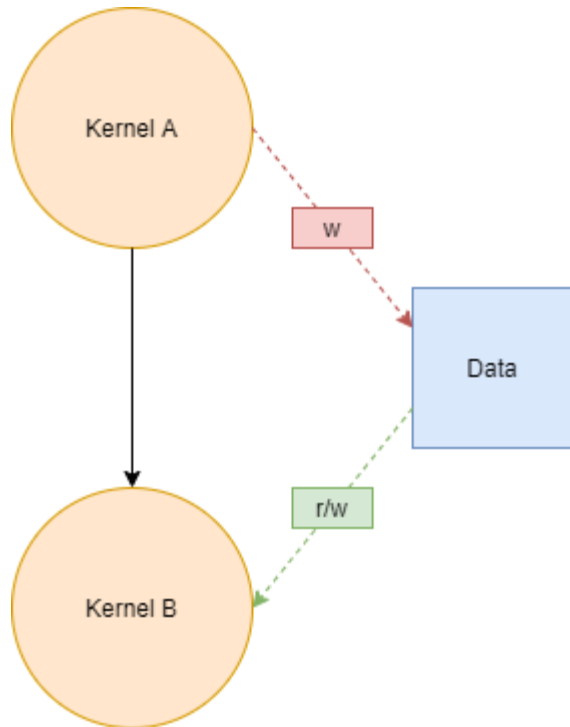
// Shared memory can be accessed on host as well as device
for (auto i = 0; i < n; ++i)
    shared_ptr[i] = i;

// ...
// Do some computation on device
// ...

// Free allocated data
sycl::free(shared_ptr, myQueue);
```

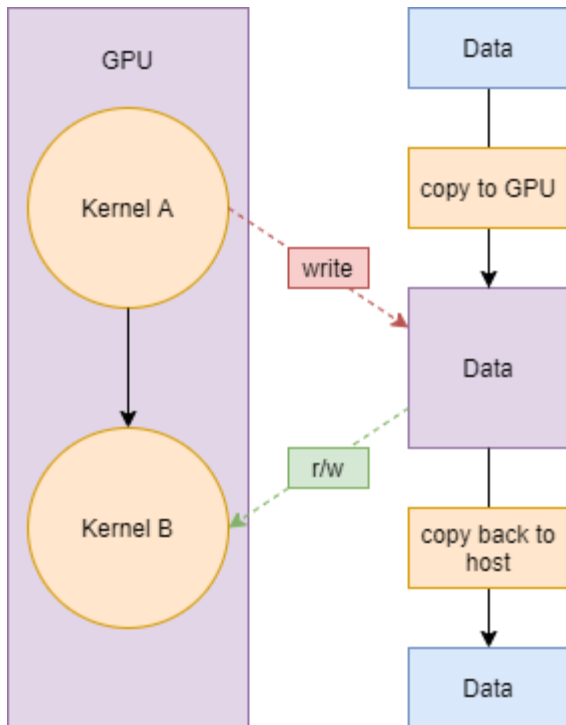
- Shared memory is accessible on host and device.
- Performance of shared memory accesses may be poor depending on platform.

CREATING DEPENDENCIES



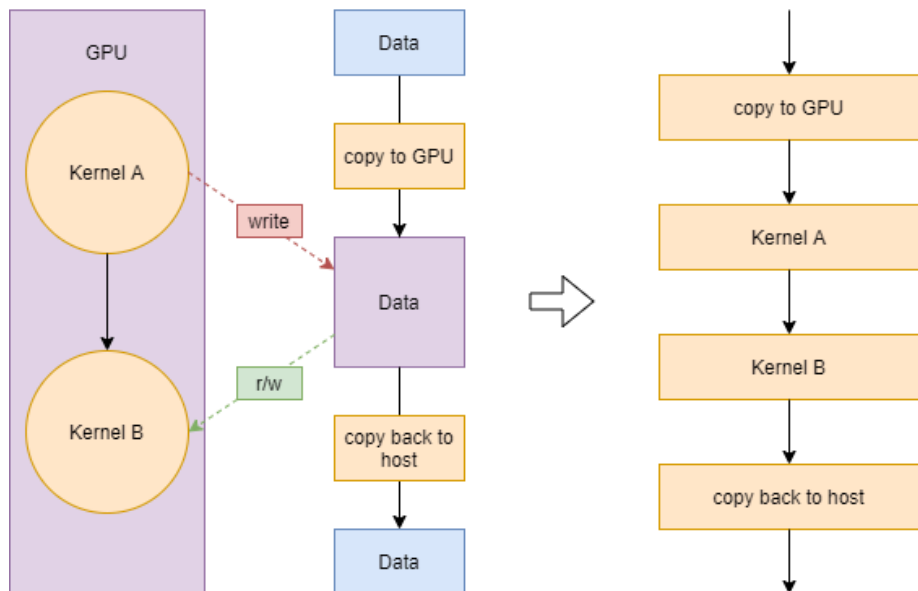
- Kernel A first writes to the data
- Kernel B then reads from and writes to the data
- This creates a read-after-write (RAW) relationship
- There must be a dependency created between Kernel A and Kernel B

MOVING DATA



- Here both kernel functions are enqueued to the same device, in this case a GPU
- The data must be copied to the GPU before the Kernel A is executed
- The data must remain on the GPU for Kernel B to be executed
- The data must be copied back to the host after Kernel B has executed

DATA FLOW



- Combining kernel function dependencies and the data movement dependencies we have a final data flow graph
- This graph defines the order in which all commands must execute in order to maintain consistency
- In more complex data flow graphs there may be multiple orderings which can achieve the same consistency

DATA FLOW WITH USM

```
1  auto devPtr =  
    sycl::malloc_device<int>(1, q);  
2  
3  auto e1 = q.memcpy(devPtr, data,  
    sizeof(int));  
4  auto e2 = q.single_task(e1,  
5      [=]() {  
6          *devPtr = /* some computation  
7              */  
8      });  
9  auto e3 = q.single_task(e2,  
10     [=]() {  
11         *devPtr = /* some computation  
12             */  
13     });  
14  auto e4 = q.memcpy(data, devPtr,  
    sizeof(int), e3);  
15  e4.wait();  
16  
17  sycl::free(devPtr, q);
```

- The USM data management model data model is prescriptive
- Dependencies are defined explicitly by passing around event objects
- Data movement is performed explicitly by enqueueing memcpy operations
- The user is responsible for ensuring data dependencies and consistency are maintained

DATA FLOW WITH USM

```
1  auto devPtr =  
    sycl::malloc_device<int>(1, q);  
2  
3  auto e1 = q.memcpy(devPtr, data,  
    sizeof(int));  
4  auto e2 = q.single_task(e1,  
5      [=]() {  
6          *devPtr = /* some computation  
7              */  
8      });  
9  auto e3 = q.single_task(e2,  
10     [=]() {  
11         *devPtr = /* some computation  
12             */  
13     });  
14  auto e4 = q.memcpy(data, devPtr,  
    sizeof(int), e3);  
15  e4.wait();  
16  
17  sycl::free(devPtr, q);
```

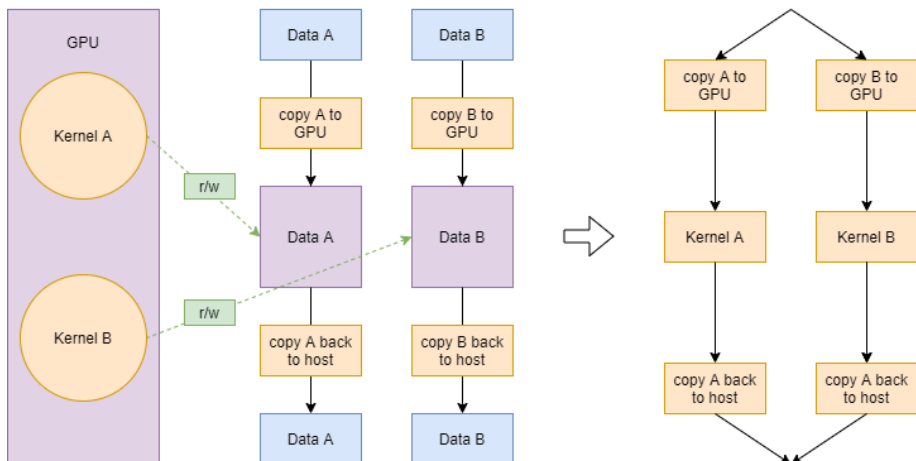
- Each command enqueued to the queue produces an event object which can be used to synchronize with the completion of that command
- Passing those event objects when enqueueing other commands creates dependencies

DATA FLOW WITH USM

```
1  auto devPtr =
    sycl::malloc_device<int>(1, q);
2
3  auto e1 = q.memcpy(devPtr, data,
    sizeof(int));
4  auto e2 = q.single_task(e1,
5      [=]() {
6          *devPtr = /* some computation
7              */
8      });
9  auto e3 = q.single_task(e2,
10     [=]() {
11         *devPtr = /* some computation
12             */
13     });
14  auto e4 = q.memcpy(data, devPtr,
    sizeof(int), e3);
15  e4.wait();
16
17  sycl::free(devPtr, q);
```

- The `memcpy` member functions are used to enqueue data movement commands, moving the data to the GPU and then back again

CONCURRENT DATA FLOW



- If two kernels are accessing different memory regions then there is no dependency between them
- In this case the two kernels and their respective data movement are independent
- By default queues are out-of-order which means that these commands can execute in any order
- They could also execute concurrently if the target device is able to do so

CONCURRENT DATA FLOW WITH USM

```
1  auto devPtrA = sycl::malloc_device<int>(1, q);
2  auto devPtrB = sycl::malloc_device<int>(1, q);
3
4  auto e1 = q.memcpy(devPtrA, dataA, sizeof(int));
5  auto e2 = q.memcpy(devPtrB, dataB, sizeof(int));
6
7  auto e3 = q.single_task(e1,
8      [=]() { *devPtrA = /* some computation */ });
9
10 auto e4 = q.single_task(e2,
11     [=]() { *devPtrB = /* some computation */ });
12
13 auto e5 = q.memcpy(dataA, devPtrA, sizeof(int), e3);
14 auto e6 = q.memcpy(dataB, devPtrB, sizeof(int), e4);
15
16 e5.wait();
17 e6.wait();
18
19 sycl::free(devPtrA, q);
20 sycl::free(devPtrB, q);
```

- Dependencies are defined explicitly

It creates dependencies between kernel functions but we do create

CONCURRENT DATA FLOW WITH USM

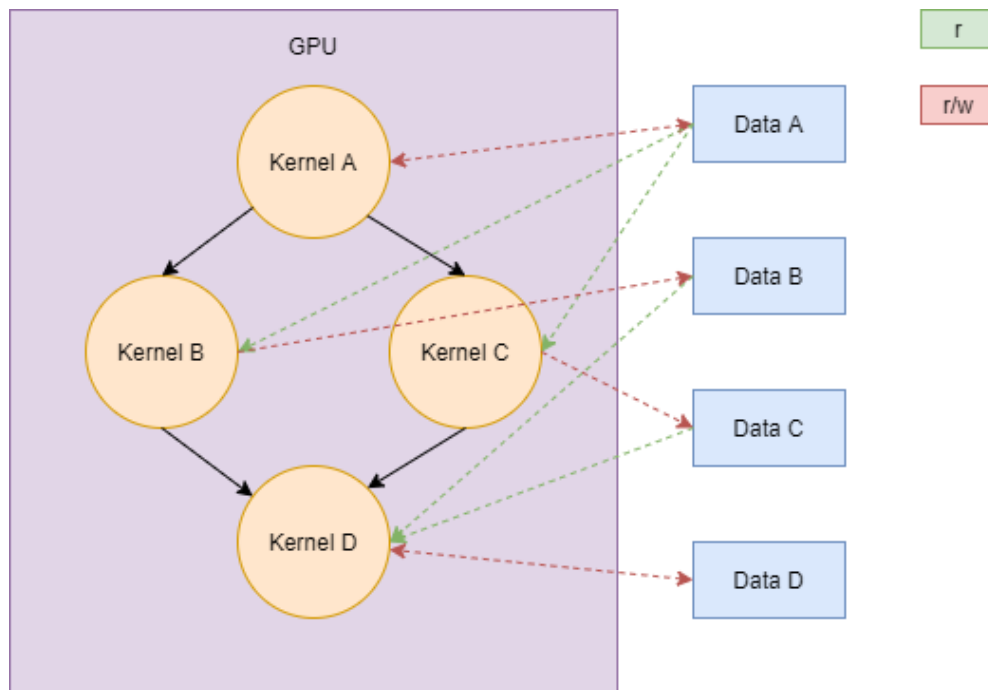
```
1  auto devPtrA = sycl::malloc_device<int>(1, q);
2  auto devPtrB = sycl::malloc_device<int>(1, q);
3
4  auto e1 = q.memcpy(devPtrA, dataA, sizeof(int));
5  auto e2 = q.memcpy(devPtrB, dataB, sizeof(int));
6
7  auto e3 = q.single_task(e1,
8      [=]() { *devPtrA = /* some computation */ });
9
10 auto e4 = q.single_task(e2,
11     [=]() { *devPtrB = /* some computation */ });
12
13 auto e5 = q.memcpy(dataA, devPtrA, sizeof(int), e3);
14 auto e6 = q.memcpy(dataB, devPtrB, sizeof(int), e4);
15
16 e5.wait();
17 e6.wait();
18
19 sycl::free(devPtrA, q);
20 sycl::free(devPtrB, q);
```

- The dependencies of each chain of commands is independent of the other, kernels and their respective copies collectively can be executed in any order.

QUESTIONS

EXERCISE

Code_Exercises/Managing_Data/source



Put together what you've seen here to create the above diamond data flow graph.

